

MODUL PRAKTIK/PRAKTIKUM

IF451 – Advance Web Programming

PROGRAM SARJANA S1

FAKULTAS TEKNIK DAN INFORMATIKA

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS MULTIMEDIA NUSANTARA

Gedung B Lantai 5, Kampus UMN

Jl. Scientia Boulevard, Gading Serpong, Tangerang, Banten-15811 Indonesia

Telp: +62-21.5422.0808, web: <http://www.umn.ac.id>



Modul Week 13

1. Lanjutkan project minggu lalu. Pada **Week12_NIMtanpa0**, buat React app dengan menggunakan vite dengan nama project **frontend**. Instal semua dependencies yang diperlukan.

```
npm create vite@latest frontend
cd frontend
npm install
npm install tailwindcss @tailwindcss/vite
npm install axios zustand react-router
```

2. Setup tailwind css :
 - Pada **vite.config.js**, tambahkan plugins tailwind css

```
frontend > vite.config.js > ...
1  import { defineConfig } from 'vite'
2  import react from '@vitejs/plugin-react'
3  import tailwindcss from '@tailwindcss/vite'
4
5  // https://vite.dev/config/
6  export default defineConfig({
7    | plugins: [react(), tailwindcss()],
8  })
```

- Pada **src/index.css**, import tailwind css.

```
frontend > src > # index.css
1  @import "tailwindcss";
```

Axios

1. Pada folder **frontend/src/lib**, buat file **api.js**. Kita akan membuat axios instance untuk fetch API ke backend Express.js. Axios instance inilah yang akan dipakai setiap kali kita melakukan fetch API ke backend. withCredentials true membuat cookie ikut terkirim (req.cookies).

```
frontend > src > lib > JS api.js > ...
1  import axios from "axios";
2
3  const api = axios.create({
4    |   baseURL: "http://localhost:5001/",
5    |   withCredentials: true, // Cookie ikut terkirim
6  });
7
8  export default api;
```

Zustand

1. Pada folder **frontend/src/lib**, buat file **useAuthStore.js** yang berguna untuk menyimpan state global (zustand) yang akan dipakai untuk menyimpan data user yang berguna untuk autentikasi :

```

frontend > src > lib > JS useAuthStore.js > ...
1  import { create } from "zustand";
2
3  const useAuthStore = create((set) => ({
4    // Menyimpan data user yang sedang login
5    // Struktur user = {
6    //   id: integer,
7    //   name: string,
8    //   role: string
9    // }
10   user: null,
11   // Function menyimpan data user ke dalam store
12   setUser: (user) => set({ user }),
13   // Function untuk menghapus data user dari store (logout)
14   logout: () => set({ user: null }),
15 });
16
17 export default useAuthStore;

```

Routing Frontend

1. Pada **frontend/src/Main.jsx**, bungkus app dengan **BrowserRouter** dari library **react-router**.

```

frontend > src > main.jsx
1  import { StrictMode } from "react";
2  import { createRoot } from "react-dom/client";
3  import "./index.css";
4  import App from "./App.jsx";
5  import { BrowserRouter } from "react-router";
6
7  createRoot(document.getElementById("root")).render(
8    <StrictMode>
9      <BrowserRouter>
10       | <App />
11     </BrowserRouter>
12   </StrictMode>
13 );

```

2. Buatlah folder **frontend/src/pages**, lalu didalamnya, buat file **Login.jsx**, **Home.jsx**, **TaskSubmission.jsx**, **EditTask.jsx**, dan **AddTask.jsx**. Pada **App.jsx**, tambahkan route untuk masing masing page.

```

frontend > src > App.jsx > ...
2  import Login from "./pages/Login.jsx";
3  import Home from "./pages/Home.jsx";
4  import Task from "./pages/Task.jsx";
5  import TaskSubmission from "./pages/TaskSubmission.jsx";
6  import EditTask from "./pages/EditTask.jsx";
7  import AddTask from "./pages/AddTask.jsx";
8
9  function App() {
10   return (
11     <Routes>
12       <Route path="/" element={<Home />} />
13       <Route path="/login" element={<Login />} />
14       <Route path="/task/submission/:id" element={<TaskSubmission />} />
15       <Route path="/task/add" element={<AddTask />} />
16       <Route path="/task/edit/:id" element={<EditTask />} />
17       <Route path="/task/:id" element={<Task />} />
18       <Route path="/unauthorized" element={<div>Unauthorized Access</div>} />
19     </Routes>
20   );
21 }
22
23 export default App;

```

3. Tidak semua page bisa diakses oleh publik. Beberapa hanya bisa diakses oleh user yang telah login. Oleh karena itu kita memerlukan function yang mengecek apakah user sudah login atau belum. Selain itu, kita juga perlu membatasi halaman yang bisa diakses mahasiswa ataupun admin.

Buatlah folder **frontend/src/components**, dan buatlah file **ProtectedRoute.jsx** di dalamnya. ProtectedRoute akan menerima parameter children (component React) dan allowedRoles (String[]).

```
frontend > src > components > ProtectedRoute.jsx > ...
1  import { Navigate } from "react-router";
2  import { useEffect, useState } from "react";
3  import useAuthStore from "../lib/useAuthStore";
4  import api from "../lib/api";
5
6  function ProtectedRoute({ children, allowedRoles = [] }) {
7    // Data user dari global state (zustand)
8    const user = useAuthStore((state) => state.user);
9    // Function untuk menyimpan data user ke global state
10   const setUser = useAuthStore((state) => state.setUser);
11   const [loading, setLoading] = useState(true);
12
13   useEffect(() => {
14     const checkAuth = async () => {
15       try {
16         // Zustand menyimpan data user di memory saja,
17         // jadi saat halaman di refresh, data user akan hilang
18         // jadi kita perlu cek ke backend apakah user masih login atau tidak
19         // Jika masih, kita ambil data user nya
20
21         // Cek apakah data user sudah ada di zustand
22         if (!user) {
23           // Jika tidak ada, panggil endpoint /api/auth/me
24           // untuk mendapatkan data user yang sedang login
25           const res = await api.get("/api/auth/me");
26           if (res.data) {
27             setUser(res.data.user);
28           }
29         }
30       } catch (err) {
31         console.error("Auth check failed:", err);
32       } finally {
33         setLoading(false);
34       }
35     };
36
37     checkAuth();
38   }, [user, setUser]);
39
40   if (loading) {
41     return <p>Loading...</p>;
42   }
43
44   // Jika tidak ada user, redirect ke halaman login
45   if (!user) {
46     return <Navigate to="/login" replace />;
47   }
48
49   // Jika tidak ada allowedRoles, berarti semua role boleh mengakses
50   // Jika ada allowedRoles dan role user tidak ada di allowedRoles,
51   // redirect ke halaman unauthorized
52   if (allowedRoles.length > 0 && !allowedRoles.includes(user.role)) {
53     return <Navigate to="/unauthorized" replace />;
54   }
55
56   return children;
57 }
58
59 export default ProtectedRoute;
```

4. Update **App.jsx**, tambahkan function ProtectedRoute untuk setiap route yang memerlukan autentikasi terlebih dahulu.

```
frontend > src > App.jsx > ...
1  import { Routes, Route } from "react-router";
2  import Login from "../pages/Login.jsx";
3  import Home from "../pages/Home.jsx";
4  import Task from "../pages/Task.jsx";
5  import TaskSubmission from "../pages/TaskSubmission.jsx";
6  import ProtectedRoute from "../components/ProtectedRoute.jsx";
7  import EditTask from "../pages/EditTask.jsx";
8  import AddTask from "../pages/AddTask.jsx";
9
10 function App() {
11   return (
12     <Routes>
13       <Route path="/" element={
14         <ProtectedRoute>
15           <Home />
16         </ProtectedRoute>
17       } />
18     </Route>
19
20     <Route path="/login" element={<Login />} />
21
22     <Route path="/task/submission/:id" element={
23       <ProtectedRoute allowedRoles={["mahasiswa"]}>
24         <TaskSubmission />
25       </ProtectedRoute>
26     } />
27
28     <Route path="/task/add" element={
29       <ProtectedRoute>
30         <AddTask />
31       </ProtectedRoute>
32     } />
33
34     <Route path="/task/edit/:id" element={
35       <ProtectedRoute allowedRoles={["admin"]}>
36         <EditTask />
37       </ProtectedRoute>
38     } />
39
40     <Route path="/task/:id" element={
41       <ProtectedRoute>
42         <Task />
43       </ProtectedRoute>
44     } />
45
46     <Route path="/unauthorized" element={<div>Unauthorized Access</div>} />
47   </Routes>
48 );
49 }
50
51 export default App;
```

Login.jsx

Pada **Login.jsx**, user akan diminta untuk memasukkan email dan password. Data input dari user akan dikirim ke backend dan akan diperiksa kevalidannya. Jika email dan password tidak match (salah) maka catch error akan menangkap error tersebut. Jika email dan password match, maka data user yang dikirim oleh backend akan disimpan di state Zustand dan akan di navigate ke page Home ("/").

```

frontend > src > pages > Login.jsx > ...
1  import { useState } from "react";
2  import api from "../lib/api";
3  import { useNavigate } from "react-router";
4  import useAuthStore from "../lib/useAuthStore";
5
6  function Login() {
7    const [email, setEmail] = useState("");
8    const [password, setPassword] = useState("");
9    const [error, setError] = useState(null);
10   const setUser = useAuthStore((state) => state.setUser);
11   const navigate = useNavigate();
12
13   const handleLogin = async (e) => {
14     // Agar halaman tidak reload
15     e.preventDefault();
16     try {
17       // Panggil API login
18       const res = await api.post("/api/auth/login", { email, password });
19       // Jika email dan password match,
20       if (res.data) {
21         // simpan data user ke global state (zustand)
22         setUser(res.data.user);
23       }
24       navigate("/");
25     } catch (err) {
26       console.error("Login failed:", err);
27       setError(err.response.data?.message);
28     }
29   };
30
31   return (
32     <>
33     <h1>Login</h1>
34     {error && <p>{error}</p>}
35     <form onSubmit={handleLogin}>
36       <label>Email: </label>
37       <input
38         type="email"
39         name="email"
40         onChange={(e) => setEmail(e.target.value)}
41         required
42       />
43       <br />
44       <label>Password: </label>
45       <input
46         type="password"
47         name="password"
48         onChange={(e) => setPassword(e.target.value)}
49         required
50       />
51       <br />
52       <button type="submit">Login</button>
53     </form>
54   </>
55   );
56 }
57
58 export default Login;

```

Home.jsx

Pada **Home.jsx**, kita akan menampilkan seluruh tugas yang ada beserta dengan button logout. Jika yang sedang login adalah admin, maka akan terdapat button add task. Tambahkan kode berikut ke Home.jsx.

```
frontend > src > pages > Home.jsx > ...
1  import { useState, useEffect } from "react";
2  import api from "../lib/api";
3  import useAuthStore from "../lib/useAuthStore";
4  import { useNavigate } from "react-router";
5
6  function Home() {
7    const [tasks, setTasks] = useState([]);
8    const [loading, setLoading] = useState(true);
9    const [error, setError] = useState(null);
10   const user = useAuthStore((state) => state.user);
11   const logout = useAuthStore((state) => state.logout);
12   const navigate = useNavigate();
13
14   useEffect(() => {
15     const fetchTasks = async () => {
16       try {
17         setLoading(true)
18         // fetch seluruh tugas yang ada
19         const res = await api.get("/api/task");
20         // set ke state React
21         setTasks(res.data);
22         setLoading(false);
23       } catch (err) {
24         console.error("Failed to fetch tasks:", err);
25         setError("Failed to fetch tasks");
26         setLoading(false);
27       }
28     };
29
30     fetchTasks();
31     console.log("User info:", user);
32   }, []);
33
34   const handleLogout = async () => {
35     try {
36       setLoading(true);
37       // fetch endpoint logout ke backend
38       await api.get("/api/auth/logout");
39       // panggil fungsi logout dari zustand store
40       // fungsi ini akan menghapus data user dari state global (zustand)
41       logout();
42       // redirect user ke halaman login
43       navigate("/login");
44     } catch (err) {
45       console.error("Logout failed:", err);
46       setError("Logout failed");
47     }
48   };
49 }
```

```

50   return (
51     <
52       <h1>Home Page</h1>
53       {loading ? (
54         <p>Loading...</p>
55       ) : (
56         <
57           <p>Welcome, {user && user.name}!</p>
58
59           {user.role === "admin" && (
60             <button onClick={() => navigate("/task/add")}>
61               Add Task
62             </button>
63           )}
64           {error && <p>error</p>}
65           {tasks &&
66             tasks.map((task) => (
67               <div key={task.id} onClick={() => navigate("/task/" + task.id)}>
68                 <h2>Tugas : {task.title}</h2>
69               </div>
70             )}
71         </>
72       )}
73       <button onClick={handleLogout}>Logout</button>
74     </>
75   );
76 }
77
78 export default Home;

```

Task.jsx

Halaman ini akan menampilkan detail dari tugas seperti judul tugas, deskripsi, dan file pendukung tugas. Pada mahasiswa, jika ia telah mengumpulkan tugas, maka akan memunculkan semua file submisi yang telah ia kumpulkan. Terdapat juga button untuk delete semua submission dan edit submission. Pada admin, admin dapat melihat semua submisi dari mahasiswa.

```

frontend > src > pages > Task.jsx > ...
1   import { useState, useEffect } from "react";
2   import api from "../lib/api";
3   import useAuthStore from "../lib/useAuthStore";
4   import { useParams } from "react-router";
5   import { useNavigate } from "react-router";
6
7   const Task = () => {
8     // ambil id dari parameter URL
9     const { id } = useParams();
10    const [task, setTask] = useState(null);
11    const [loading, setLoading] = useState(true);
12    const [error, setError] = useState(null);
13    const user = useAuthStore((state) => state.user);
14    const navigate = useNavigate();
15    const BASE_URL = "http://localhost:5001/";
16
17    useEffect(() => {
18      const fetchTask = async () => {
19        try {
20          setLoading(true);
21          // fetch data task berdasarkan id
22          const res = await api.get("/api/task/" + id);
23          setTask(res.data);
24          setLoading(false);
25        } catch (err) {
26          console.error("Failed to fetch task:", err);
27          setError("Failed to fetch task");
28          setLoading(false);
29        }
30      };
31
32      fetchTask();
33    }, []);
34

```

```

35 const handleDeleteSubmission = async () => {
36   try {
37     setLoading(true);
38     // Delete semua submission terkait task ini
39     await api.delete("/api/task/submission/" + id);
40     // Refresh task data setelah penghapusan
41     const res = await api.get("/api/task/" + id);
42     setTask(res.data);
43     setLoading(false);
44   } catch (err) {
45     console.error("Failed to delete submissions:", err);
46     setError("Failed to delete submissions");
47   }
48 };
49
50 const handleSubmission = () => {
51   navigate(`/task/submission/${id}`);
52 };
53
54 return (
55   <div>
56     {loading ? (
57       <p>Loading...</p>
58     ) : (
59       <div>
60         <h1>Task Detail</h1>
61         {error && <p>{error}</p>}
62         <h1>Judul Tugas: {task.title}</h1>
63         <p>Deskripsi Tugas: {task.description}</p>
64         <p>
65           File Pendukung:{" "}
66           <a href={BASE_URL + task.filePath} target="_blank" rel="noopener noreferrer">
67             {task.filePath.split("-").slice(2).join("-")}
68           </a>
69         </p>
70       </div>
71     )}
72   </div>
73
74   <div>
75     // Render untuk role mahasiswa
76     user.role === "mahasiswa" &&
77     (task.TaskSubmissions && task.TaskSubmissions.length > 0 ? (
78       <div>
79         <h2>Your Submissions:</h2>
80         <button onClick={handleDeleteSubmission}>
81           Delete All Submission
82         </button>
83         <br />
84         <a href={`/task/submission/${id}`}>Edit Submission</a>
85         <ul>
86           {/* list task submission milik user yang sedang login */}
87           {task.TaskSubmissions.map((submission) => (
88             <li key={submission.id}>
89               <a
90                 href={submission.filePath}
91                 target="_blank"
92                 rel="noopener noreferrer"
93               >
94                 {submission.filePath.split("-").slice(2).join("-")}
95               </a>
96             </li>
97           ))}
98         </ul>
99       </div>
100     ) : (
101       <button onClick={handleSubmission}>Add submission</button>
102     ))
103   </div>
104 )

```

```

101     /* Untuk role admin */
102     {user.role === "admin" && (
103         <>
104         <a href={`/task/edit/${id}`}>Edit Task</a>
105         <h2>All Submissions:</h2>
106         /* Menampilkan seluruh submission dari semua mahasiswa */
107         {task.TaskSubmissions && task.TaskSubmissions.length > 0 ? (
108             Object.entries(
109                 task.TaskSubmissions.reduce((acc, submission) => {
110                     if (!acc[submission.userId]) {
111                         acc[submission.userId] = [];
112                     }
113                     acc[submission.userId].push(submission);
114                     return acc;
115                 }, {})
116             ).map(([userId, submissions]) => (
117                 <div key={userId}>
118                     <h3>User ID: {userId}</h3>
119                     <ul>
120                         {submissions.map((submission) => (
121                             <li key={submission.id}>
122                                 <a
123                                     href={BASE_URL + submission.filePath}
124                                     target="_blank"
125                                     rel="noopener noreferrer"
126                                 >
127                                     {submission.filePath.split("-").slice(2).join("-")}
128                                 </a>
129                             </li>
130                         ))}
131                     </ul>
132                 </div>
133             ))
134             : (
135                 <p>No submissions yet.</p>
136             )}
137         </>
138     )}
139 </>
140 )}
141 </>
142 );
143 };
144
145 export default Task;

```

Tugas

Lengkapilah **AddTask.jsx**, **EditTask.jsx**, dan **TaskSubmission.jsx** dengan ketentuan sebagai berikut:

- AddTask.jsx: Page yang hanya bisa diakses oleh admin dimana admin dapat membuat task baru.
- EditTask.jsx: Page yang hanya bisa diakses oleh admin dimana admin dapat mengedit task yang ada.
- TaskSubmission: Page yang hanya bisa diakses oleh mahasiswa dimana mahasiswa dapat mengumpulkan tugas ataupun mengedit submission yang telah ia kumpulkan.

Tambahkan juga desain pada setiap page. Desain dibebaskan sekreatif mungkin.